TEXT SUMMARIZATION USING DEEP LEARNING

A Mini Project Report

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

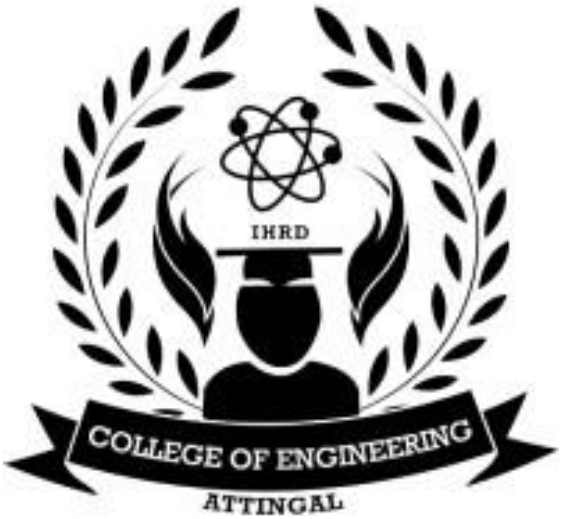
in

Computer Science Engineering

(Artificial Intelligence and Machine Learining)

by

APARNA N (CEA24AM015)



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

COLLEGE OF ENGINEERING ATTINGAL

KERALA

TABLE OF CONTENTS

SERIAL NUMBER	TITLE	PAGE NUMBER
01	Intoduction	03
02	Problem Statements and Objectives	04
03	System Design and Architeture	05-07
04	Implemetation	08-13
05	Result	14-15
06	Conclusion	16

INTRODUCTION

The rapid growth of digital information has created a large volume of textual data from news, research papers, websites, and reports. Reading lengthy documents is time-consuming. **Text summarization**, an important Natural Language Processing (NLP) task, automatically generates concise summaries while preserving the key information and meaning of the original text.Summarization can be **extractive**, where important sentences are selected from the source, or **abstractive**, where new sentences are generated based on the content. This project focuses on abstractive summarization using the **T5-small Transformer model**, which is fine-tuned on the **CNN/DailyMail dataset** containing news articles and human-written summaries.The system is implemented using **Python, PyTorch, Hugging Face Transformers, and Datasets**. The model performance is primarily evaluated using **ROUGE-1, ROUGE-2, and ROUGE-L**, along with training and validation loss. A **Streamlit interface** is also developed to allow users to generate summaries from their own text. Long documents can be divided into smaller chunks and summarized individually.Overall, the project demonstrates the application of **Deep Learning and Transformer-based NLP techniques** to generate concise and meaningful summaries through an end-to-end system involving preprocessing, model fine-tuning, evaluation, and deployment.

PROBLEM STATEMENT AND OBJECTIVES

Problem Statement:

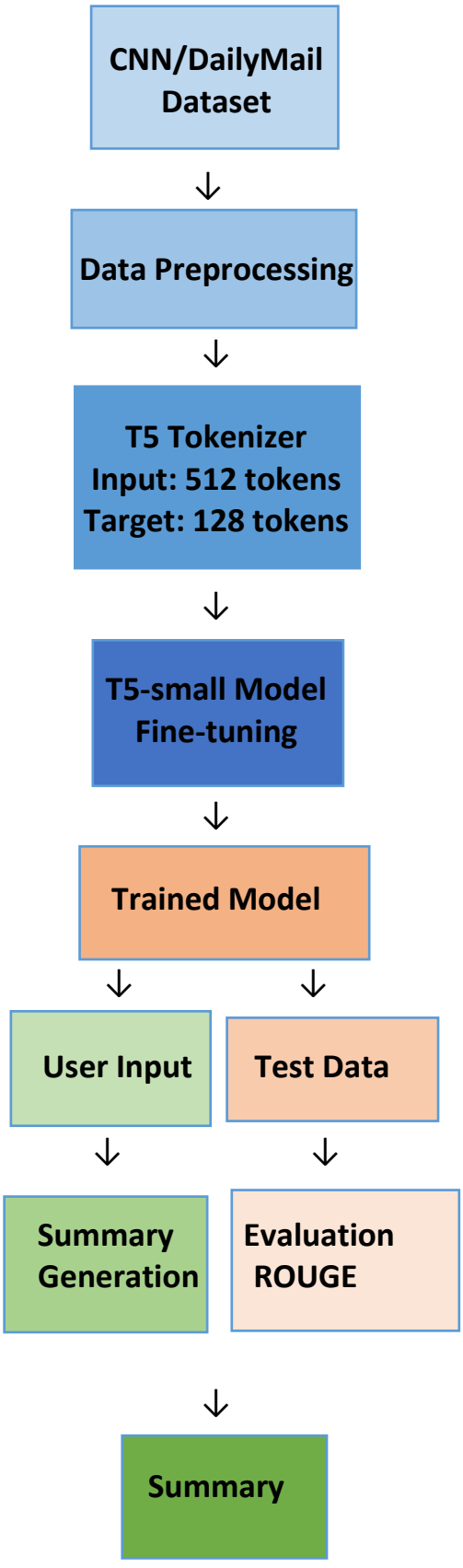
The rapid growth of digital information has made it difficult and time-consuming to read lengthy documents and identify their important information. Traditional extractive methods may produce summaries by selecting existing sentences, but they may lack coherence and fail to generate concise, natural summaries. Therefore, there is a need for an automated system that can understand the content of a document and generate a meaningful summary.This project addresses this problem by developing an **abstractive text summarization system using the Transformer-based T5-small model**. The system is trained using the **CNN/DailyMail dataset** and evaluated using appropriate summarization metrics.

Objectives:

- To develop an **abstractive text summarization system** using the T5 Transformer model.To preprocess and utilize the **CNN/DailyMail dataset** for model training and evaluation.
- To fine-tune the **T5-small model** to generate concise summaries.To evaluate the model using **ROUGE-1, ROUGE-2, and ROUGE-L** metrics.
- To analyze training performance using **loss, precision, recall, F1-score, MSE, and RMSE**.
- To develop a **Streamlit-based interface** for generating summaries from user-provided text.To support summarization of **longer textual inputs** through text chunking.

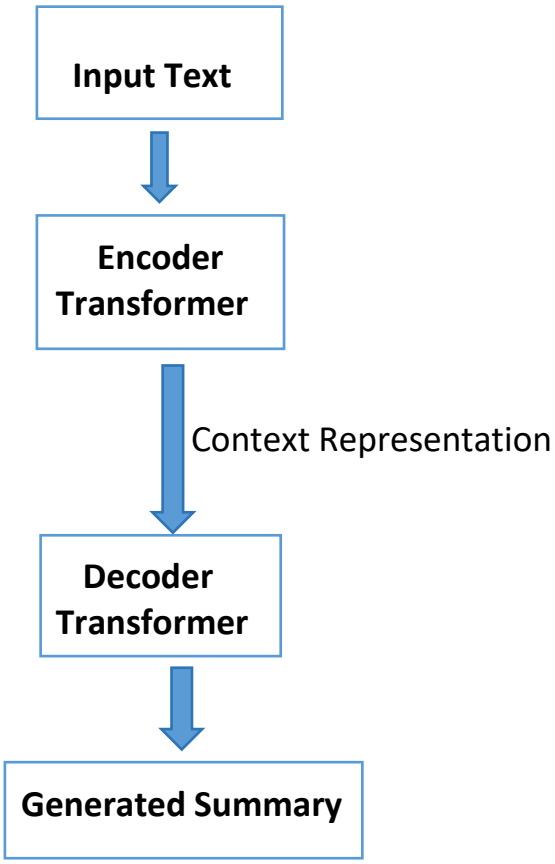
SYSTEM DESIGN AND ARCHITECTURE

The proposed system uses a **T5-small Transformer model** to generate abstractive summaries from lengthy text. The system consists of data preprocessing, tokenization, model training, evaluation, and summary generation modules.



● T5 Model Architecture

T5 follows an encoder-decoder Transformer architecture:



The encoder converts the input text into contextual representations, while the decoder generates the summary token by token.

● Text Processing

The input is formatted as:

X="summarize: "+ArticleX

The tokenizer converts the text into token IDs:

X→[x1,x2,...,xn]

where the maximum input length is **512 tokens** and the target summary length is **128 tokens**.

For long documents, the text is divided into smaller chunks:

```
                  ____
                 | C1 | --> S1 ---
                 ____              |
Long Document --> | C2 | --> S2 ---|------------>FINAL SUMMARY
                 ____              |
●                | C3 | --> S3 ---
                 ____
```

- ## Training

The model learns by minimizing the difference between the generated summary and the reference summary using cross-entropy loss:

$$L = -\sum_{t=1}^{T} log\ P(y_t | y_{<t}, X)$$

where **X** is the input article and $\mathbf{y_t}$ is the target summary token.

- ## Evaluation

The generated summaries are compared with reference summaries using ROUGE:

$$\textbf{ROUGE-1} = \frac{\textbf{Overlapping Unigrams}}{\textbf{Reference Unigrams}}$$

$$\textbf{ROUGE-2} = \frac{\textbf{Overlapping Bigrams}}{\textbf{Reference Bigrams}}$$

$$\textbf{ROUGE-L} = \frac{\textbf{LCS-based overlap}}{\textbf{Reference Length}}$$

Additional token-level metrics such as **accuracy, precision, recall, F1-score, MSE, and RMSE** are used as supplementary measures.

- ## Technology Architecture

Python

↓

PyTorch + Hugging Face Transformers

↓

T5-small

↓

CNN/DailyMail Dataset

↓

ROUGE Evaluation

↓

Streamlit Interface

IMPLEMENTATION

```
!pip install -q transformers datasets evaluate
rouge_score
sentencepiece accelerate matplotlib pandas
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import transformers
import datasets
import evaluate
print("PyTorch version:", torch.__version__)
print("Transformers version:",
transformers.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
print("GPU:", torch.cuda.get_device_name(0))
from datasets import load_dataset
dataset = load_dataset("abisee/cnn_dailymail",
"3.0.0")
print(dataset)
print("Training samples:", len(dataset["train"]))
print("Validation samples:",
len(dataset["validation"]))
print("Test samples:", len(dataset["test"]))
print("\nExample article:",
dataset["train"][0]["article"][:500],
"...")
print("\nReference summary:",
dataset["train"][0]["highlights"])
from transformers import AutoTokenizer
model_name = "t5-small"
tokenizer =
AutoTokenizer.from_pretrained(model_name)
print("Tokenizer loaded successfully!")
max_input_length = 512
max_target_length = 128
def preprocess_function(examples):
inputs = ["summarize: " + article for article in
examples["article"]]
targets = examples["highlights"]
model_inputs = tokenizer(inputs,
max_length=max_input_length,
truncation=True, padding="max_length")
labels = tokenizer(targets,
max_length=max_target_length,
truncation=True, padding="max_length")
model_inputs["labels"] = labels["input_ids"]
return model_inputs
print("Preprocessing function
ready.")train_dataset =
dataset["train"].select(range(10000))
validation_dataset =
dataset["validation"].select(range(1000))
test_dataset = dataset["test"].select(range(1000))
print("Training samples:", len(train_dataset))
print("Validation samples:", len(validation_dataset))
print("Test samples:", len(test_dataset))
tokenized_train =
train_dataset.map(preprocess_function,
batched=True)
tokenized_validation =
validation_dataset.map(preprocess_function,
batched=True)
tokenized_test =
test_dataset.map(preprocess_function,
batched=True)
print("Tokenization completed!")
from transformers import
T5ForConditionalGeneration
model =
T5ForConditionalGeneration.from_pretrained("t5-
small")
device = torch.device("cuda" if
torch.cuda.is_available() else "cpu")
model = model.to(device)
print("Using device:", device)
tokenized_train =
tokenized_train.remove_columns(["article",
"highlights", "id"])
tokenized_validation =
tokenized_validation.remove_columns(["article",
"highlights", "id"])
tokenized_test =
tokenized_test.remove_columns(["article",
"highlights", "id"])
print("Unnecessary raw-text columns removed.")
from transformers import DataCollatorForSeq2Seq
data_collator =
DataCollatorForSeq2Seq(tokenizer=tokenizer,
model=model)
print("Data collator created successfully.")
from transformers import
Seq2SeqTrainingArguments
```

```
training_args = Seq2SeqTrainingArguments(
output_dir="./t5-summarization",
eval_strategy="steps",
eval_steps=500,
save_steps=500,
logging_steps=100,
learning_rate=2e-5,
per_device_train_batch_size=4,
per_device_eval_batch_size=4,
num_train_epochs=1,
weight_decay=0.01,
predict_with_generate=True,
fp16=torch.cuda.is_available(),
save_total_limit=2,
report_to="none")
print("Training configuration created.")
rouge = evaluate.load("rouge")
def compute_metrics(eval_pred):
predictions, labels = eval_pred
predictions = np.asarray(predictions)
if predictions.ndim == 3:
predictions = predictions.argmax(axis=-1)
predictions = predictions.astype(np.int64)
labels = np.where(labels == -100,
tokenizer.pad_token_id,
labels).astype(np.int64)
predictions = np.clip(predictions, 0,
tokenizer.vocab_size - 1)
decoded_predictions =
tokenizer.batch_decode(predictions.tolist(),
skip_special_tokens=True)
decoded_labels =
tokenizer.batch_decode(labels.tolist(),
skip_special_tokens=True)
scores = rouge.compute(
predictions=decoded_predictions,
references=decoded_labels,
use_stemmer=True
)
return {
"rouge1": scores["rouge1"],
"rouge2": scores["rouge2"],
"rougeL": scores["rougeL"]
}
print("ROUGE evaluation function ready.")
from transformers import Seq2SeqTrainer

validation_for_training =
tokenized_validation.select(range(200))
trainer = Seq2SeqTrainer(
model=model,
args=training_args,
train_dataset=tokenized_train,
eval_dataset=validation_for_training,
processing_class=tokenizer,
data_collator=data_collator,
compute_metrics=compute_metrics
)
print("Trainer created successfully.")Train the
existing T5-small model
This keeps the original architecture and training
settings. No model redesign is introduced.
train_result = trainer.train()
print("Training completed.")
print(train_result.metrics)
from collections import Counter
def token_overlap_metrics(pred_ids, label_ids):

pred_ids = np.asarray(pred_ids, dtype=np.int64)
label_ids = np.asarray(label_ids, dtype=np.int64)
label_ids = np.where(label_ids == -100,
tokenizer.pad_token_id,
label_ids)
pred_ids = np.clip(pred_ids, 0,
tokenizer.vocab_size - 1)
total_overlap = 0
total_pred_tokens = 0
total_ref_tokens = 0
correct_tokens = 0
compared_tokens = 0
squared_error = 0.0
for pred, ref in zip(pred_ids, label_ids):
pred_tokens = [int(x) for x in pred if int(x) !=
tokenizer.pad_token_id]
ref_tokens = [int(x) for x in ref if int(x) !=
tokenizer.pad_token_id]
overlap = sum((Counter(pred_tokens) &
Counter(ref_tokens)).values())
total_overlap += overlap
total_pred_tokens += len(pred_tokens)
total_ref_tokens += len(ref_tokens)
n = min(len(pred_tokens), len(ref_tokens))
if n > 0:
pred_arr = np.asarray(pred_tokens[:n],
dtype=np.float64)
ref_arr = np.asarray(ref_tokens[:n],
dtype=np.float64)
squared_error += np.sum((pred_arr - ref_arr) ** 2)
correct_tokens += np.sum(pred_arr == ref_arr)
compared_tokens += n
precision = total_overlap / total_pred_tokens if
total_pred_tokens
```

```
else 0.0
recall = total_overlap / total_ref_tokens if
total_ref_tokens else
0.0
f1 = 2 * precision * recall / (precision + recall) if
(precision +
recall) else 0.0
accuracy = correct_tokens / compared_tokens if
compared_tokens
else 0.0
mse = squared_error / compared_tokens if
compared_tokens else 0.0
rmse = float(np.sqrt(mse))
return {
"token_precision": float(precision),
"token_recall": float(recall),
"token_f1": float(f1),
"token_accuracy": float(accuracy),
"token_id_mse": float(mse),
"token_id_rmse": rmse
}
final_test_result = trainer.predict(
tokenized_test,
max_length=128,
num_beams=4
)
predictions =
np.asarray(final_test_result.predictions)
if predictions.ndim == 3:
predictions = predictions.argmax(axis=-1)
labels = np.asarray(final_test_result.label_ids)
aux_metrics = token_overlap_metrics(predictions,
labels)
evaluation_results = {
key: float(value)
for key, value in final_test_result.metrics.items()
if isinstance(value, (int, float, np.integer,
np.floating))
}
evaluation_results.update(aux_metrics)
print("FINAL TEST EVALUATION")
print("-" * 40)
for key, value in evaluation_results.items():
print(f"{key}: {value:.6f}")
os.makedirs("./results/plots",
exist_ok=True)pd.DataFrame([evaluation_results]).
to_csv(
"./results/evaluation_results.csv",
index=False
)
import json
with open("./results/evaluation_results.json", "w")
as f:
json.dump(evaluation_results, f, indent=4)
print("Evaluation results saved.")

history = pd.DataFrame(trainer.state.log_history)
print("Available history columns:",
history.columns.tolist())
history.to_csv("./results/training_history.csv",
index=False)

plt.figure(figsize=(9, 5))
if "loss" in history.columns:
h = history.dropna(subset=["loss"])
plt.plot(h["step"], h["loss"], marker="o",
label="Training Loss")
if "eval_loss" in history.columns:
h = history.dropna(subset=["eval_loss"])
plt.plot(h["step"], h["eval_loss"], marker="o",
label="Validation
Loss")
plt.xlabel("Training Steps")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("./results/plots/loss_curve.png",
dpi=200)
plt.show()

plt.figure(figsize=(9, 5))
for metric in ["rouge1", "rouge2", "rougeL"]:
if metric in history.columns:
h = history.dropna(subset=[metric])
plt.plot(h["step"], h[metric], marker="o",
label=metric.upper())
plt.xlabel("Training Steps")
plt.ylabel("ROUGE Score")
plt.title("ROUGE Scores During Training")
plt.legend()plt.grid(True)
plt.tight_layout()
plt.savefig("./results/plots/rouge_curve.png",
dpi=200)
plt.show()

rouge_values = [
evaluation_results.get("test_rouge1",
evaluation_results.get("rouge1", 0)),
evaluation_results.get("test_rouge2",
evaluation_results.get("rouge2", 0)),
```

```
evaluation_results.get("test_rougeL",
evaluation_results.get("rougeL", 0))
]
plt.figure(figsize=(8, 5))
plt.bar(["ROUGE-1", "ROUGE-2", "ROUGE-L"],
rouge_values)
plt.ylabel("Score")
plt.title("Final Test ROUGE Scores")
plt.grid(axis="y")
plt.tight_layout()
plt.savefig("./results/plots/rouge_comparison.png",
dpi=200)
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(
["Precision", "Recall", "F1"],
[evaluation_results["token_id_mse"],
evaluation_results["token_id_rmse"]]
)
plt.ylabel("Value")
plt.title("Auxiliary Token-ID Error Metrics")
plt.grid(axis="y")
plt.tight_layout()plt.savefig("./results/plots/mse_r
mse.png", dpi=200)
plt.show()
print("All plots saved in ./results/plots/")
model_save_path =
"./final_t5_summarization_model"
model.save_pretrained(model_save_path)
tokenizer.save_pretrained(model_save_path)
print("Model and tokenizer saved successfully.")
print(os.listdir(model_save_path))
sample = dataset["test"][0]
article = sample["article"]
reference = sample["highlights"]
inputs = tokenizer(
"summarize: " + article,
return_tensors="pt",
max_length=512,
truncation=True )
inputs = {key: value.to(device) for key, value in
inputs.items()}
with torch.no_grad():
summary_ids = model.generate(
**inputs,
max_length=128,
min_length=30,
num_beams=4,
no_repeat_ngram_size=3,
early_stopping=True
[evaluation_results["token_precision"],
evaluation_results["token_recall"],
evaluation_results["token_f1"]]
)
plt.ylabel("Score")
plt.title("Auxiliary Token-Overlap Metrics")
plt.ylim(0, 1)
plt.grid(axis="y")
plt.tight_layout()
plt.savefig("./results/plots/precision_recall_f1.png",
dpi=200)
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(
["Token-ID MSE", "Token-ID RMSE"],



)
generated_summary =
tokenizer.decode(summary_ids[0],
skip_special_tokens=True)
print("ORIGINAL ARTICLE:")
print(article)
print("\nREFERENCE SUMMARY:")
print(reference)
print("\nGENERATED SUMMARY:")
print(generated_summary)

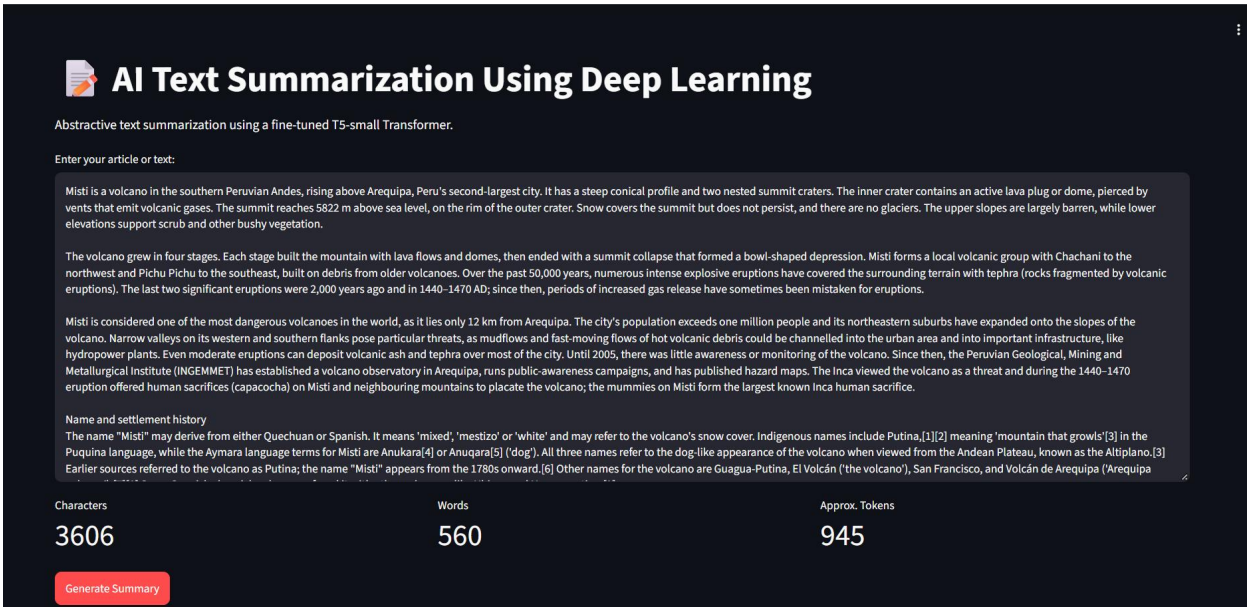
import re
def split_into_chunks(text, max_tokens=450):
sentences = re.split(r'(?<=[.!?])\s+',
text.replace("\n", "
").strip())
chunks = [] current_sentences = []
current_tokens = 0
for sentence in sentences:
if not sentence.strip():
continue
sentence_tokens = len(tokenizer.encode(sentence,
add_special_tokens=False))
if current_sentences and current_tokens +
sentence_tokens >
max_tokens:
chunks.append(" ".join(current_sentences))
current_sentences = []
current_tokens = 0
if sentence_tokens > max_tokens:
token_ids = tokenizer.encode(sentence,
```

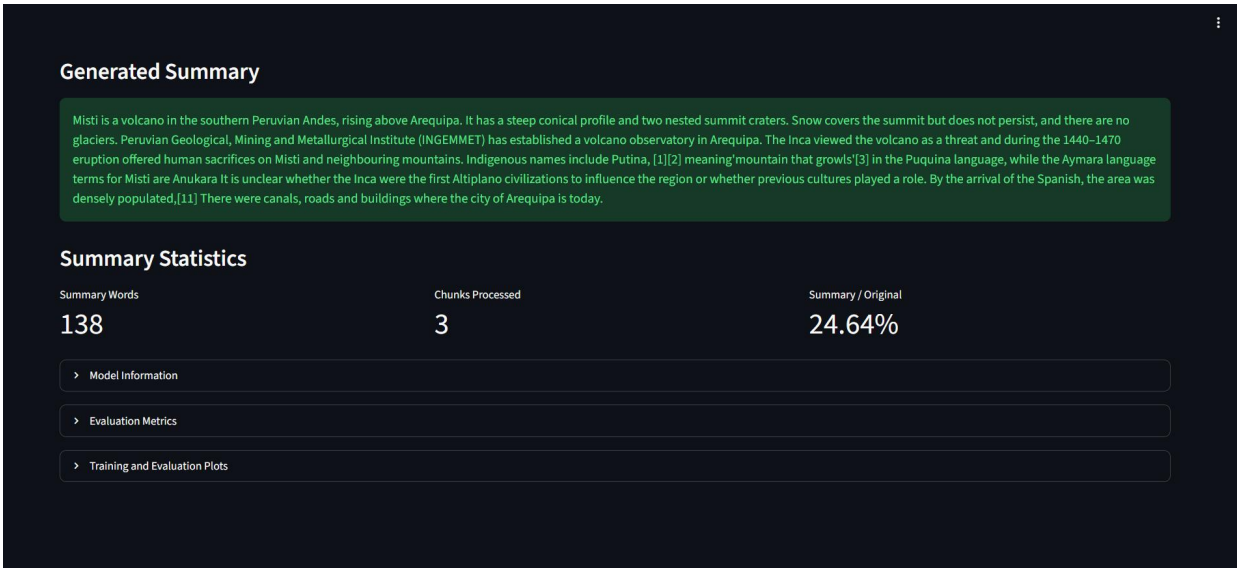
```
add_special_tokens=False)
for start in range(0, len(token_ids), max_tokens):
chunks.append(
tokenizer.decode(token_ids[start:start +
max_tokens], skip_special_tokens=True) )
continue
current_sentences.append(sentence)
current_tokens += sentence_tokens
if current_sentences:
chunks.append(" ".join(current_sentences))
return chunks
print("Long-text chunking function ready.")
import os
import re
import streamlit as st
import torch
from transformers import T5
st.set_page_config(page_title="AI Text
Summarizer", page_icon="  ",
layout="wide")DEVICE = torch.device("cuda" if
torch.cuda.is_available() else "cpu")
MODEL_PATH = "./final_t5_summarization_model"
def load_model():
if not os.path.exists(MODEL_PATH):
raise FileNotFoundError(f"Model directory not
found:
{MODEL_PATH}")
tokenizer =
AutoTokenizer.from_pretrained(MODEL_PATH)
model =
T5ForConditionalGeneration.from_pretrained(MO
DEL_PATH)
model = model.to(DEVICE)
model.eval()
return tokenizer, model
tokenizer, model = load_model()
def split_into_chunks(text, max_tokens=450):
sentences = re.split(r'(?<=[.!?])\s+',
text.replace("\n", "
").strip())
chunks, current_sentences, current_tokens = [], [],
0
for sentence in sentences:
sentence = sentence.strip()
if not sentence:
continue
sentence_tokens = len(tokenizer.encode(sentence,
add_special_tokens=False))
if current_sentences and current_tokens +
sentence_tokens >
max_tokens:
chunks.append(" ".join(current_sentences))
current_sentences, current_tokens = [], 0
if sentence_tokens > max_tokens:
token_ids = tokenizer.encode(sentence,
add_special_tokens=False)
for start in range(0, len(token_ids), max_tokens):
chunks.append(
tokenizer.decode(token_ids[start:start +
max_tokens], skip_special_tokens=True) )
continue
current_sentences.append(sentence)
current_tokens += sentence_tokens
if current_sentences:
chunks.append(" ".join(current_sentences))
return chunks
def summarize_chunk(text):
inputs = tokenizer("summarize: " + text,
return_tensors="pt",
max_length=512, truncation=True)
inputs = {key: value.to(DEVICE) for key, value in
inputs.items()}
with torch.no_grad():
summary_ids = model.generate(
**inputs,
max_length=128,
min_length=30,
num_beams=4,
length_penalty=1.0,
early_stopping=True,
no_repeat_ngram_size=3 )
return tokenizer.decode(summary_ids[0],
skip_special_tokens=True)
def summarize_long_text(text):
chunks = split_into_chunks(text)
if not chunks:
return "", 0
summaries = []
progress = st.progress(0)
status = st.empty()
for i, chunk in enumerate(chunks):
status.write(f"Summarizing part {i + 1} of
{len(chunks)}...")
summary = summarize_chunk(chunk)
if summary.strip():
summaries.append(summary)
progress.progress((i + 1) / len(chunks))
progress.empty()
status.empty()
if len(summaries) == 1:
return summaries[0], len(chunks)
combined = " ".join(summaries)
```

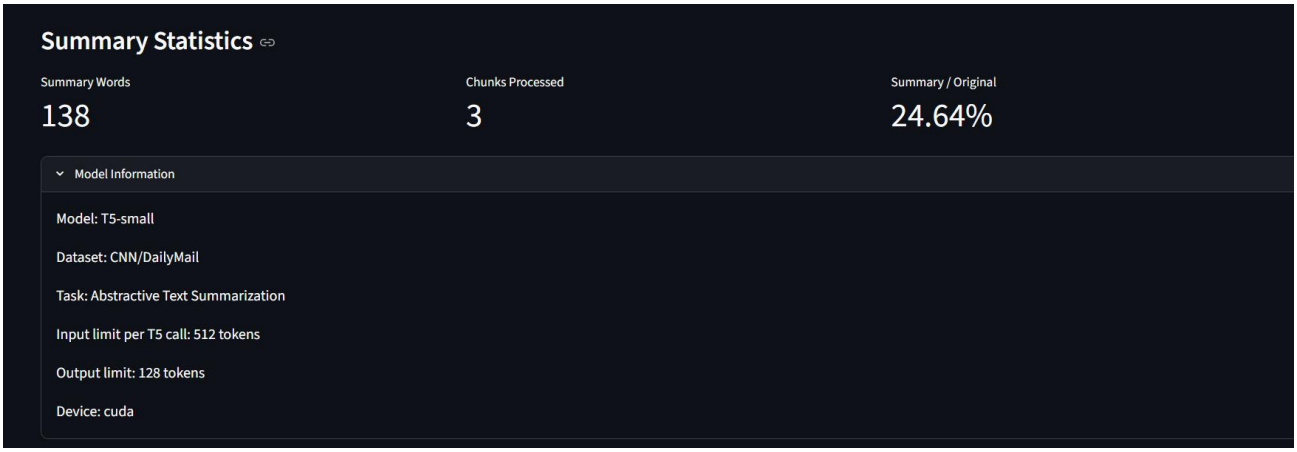
```
combined_tokens = tokenizer.encode(combined,
add_special_tokens=False)
if len(combined_tokens) <= 450:
return combined, len(chunks)
return summarize_chunk(combined), len(chunks)
st.title("   AI Text Summarization Using Deep
Learning")
st.write("Abstractive text summarization using a
fine-tuned T5-small
Transformer.")
article = st.text_area(
"Enter your article or text:",
height=400,
placeholder="Paste your article here..." )
if article.strip():
col1, col2, col3 = st.columns(3)
col1.metric("Characters", len(article))
col2.metric("Words", len(article.split()))
col3.metric("Approx. Tokens",
len(tokenizer.encode(article,
add_special_tokens=False)))
if st.button("Generate Summary", type="primary"):
if not article.strip():
st.warning("Please enter some text first.")
else:
try:
with st.spinner("Generating summary..."):
summary, chunk_count =
summarize_long_text(article)
st.subheader("Generated Summary")
st.success(summary)
original_words = len(article.split())
summary_words = len(summary.split())
ratio = summary_words / original_words if
original_words
else 0
st.subheader("Summary Statistics")
c1, c2, c3 = st.columns(3)
c1.metric("Summary Words", summary_words)
c2.metric("Chunks Processed", chunk_count)
c3.metric("Summary / Original", f"{ratio:.2%}")
except Exception as e:
st.error(f"Unable to generate the summary: {e}")
with st.expander("Model Information"):
st.write("Model: T5-small")
st.write("Dataset: CNN/DailyMail")
st.write("Task: Abstractive Text Summarization")
st.write("Input limit per T5 call: 512 tokens")
st.write("Output limit: 128 tokens")
st.write(f"Device: {DEVICE}")
with st.expander("Evaluation Metrics"):
results_file = "./results/evaluation_results.csv"
if os.path.exists(results_file):
import pandas as pd
results = pd.read_csv(results_file)
st.dataframe(results, use_container_width=True)
st.caption(
"Precision, Recall and F1 are token-overlap
auxiliary
metrics. "
"Token-ID MSE/RMSE are numeric diagnostics and
are not
semantic "
"quality measures for generated text." )
else:
st.info("Run the evaluation cells first to generate
evaluation
results.")
with st.expander("Training and Evaluation Plots"):
plot_files = [
("Training / Validation Loss",
"./results/plots/loss_curve.png"),
("ROUGE Progress",
"./results/plots/rouge_curve.png"),
("Final ROUGE Scores",
"./results/plots/rouge_comparison.png"),
("Precision / Recall / F1",
"./results/plots/precision_recall_f1.png"),
("MSE / RMSE", "./results/plots/mse_rmse.png")
]
for title, path in plot_files:
if os.path.exists(path):
st.write(f"### {title}")
st.image(path)
else:
st.info(f"{title} is not available yet.")
!pip install -q streamlit
!nohup streamlit run app.py --server.port 8501 --
server.headless true
> /content/streamlit.log 2>&1 &
import time
time.sleep(5)
!curl -I http://localhost:8501
!wget -q
https://github.com/cloudflare/cloudflared/releass/
latest/download/cloudflared-linux-amd64 -O
cloudflared
!chmod +x cloudflared

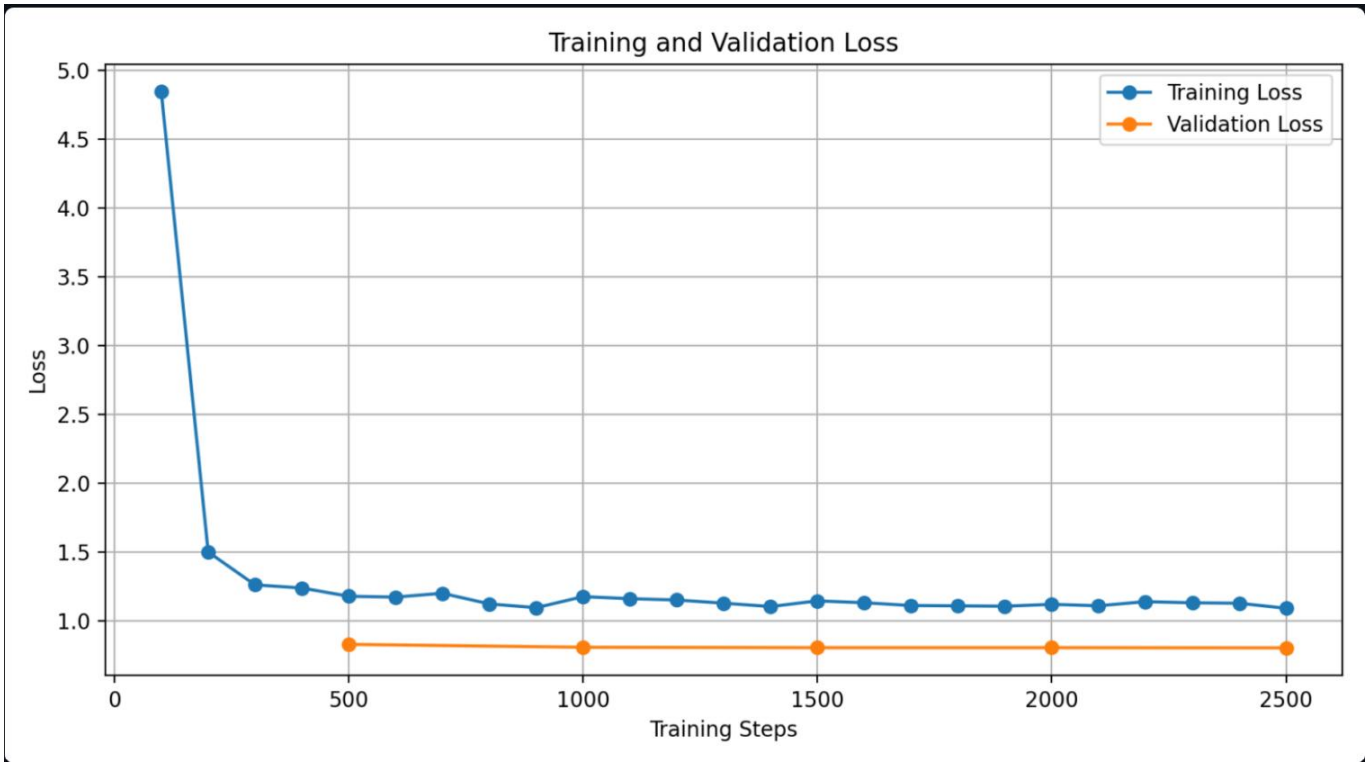
!./cloudflared tunnel --url http://localhost:8501
```

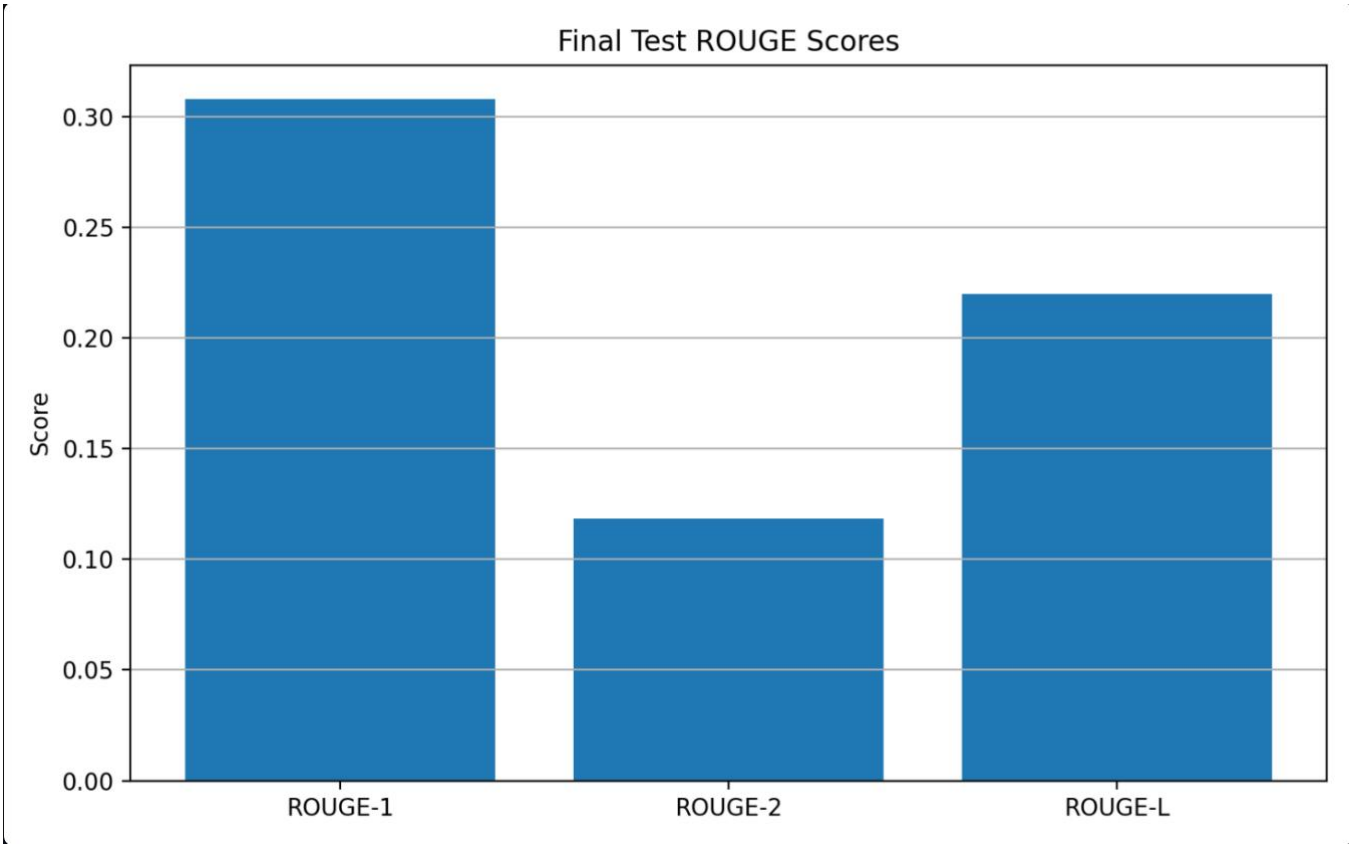
RESULT


AI Text Summarization Using Deep Learning
Abstractive text summarization using a fine-tuned T5-small Transformer.
Enter your article or text:
Misti is a volcano in the southern Peruvian Andes, rising above Arequipa, Peru's second-largest city. It has a steep conical profile and two nested summit craters. The inner crater contains an active lava plug or dome, pierced by vents that emit volcanic gases. The summit reaches 5822 m above sea level, on the rim of the outer crater. Snow covers the summit but does not persist, and there are no glaciers. The upper slopes are largely barren, while lower elevations support scrub and other bushy vegetation.
The volcano grew in four stages. Each stage built the mountain with lava flows and domes, then ended with a summit collapse that formed a bowl-shaped depression. Misti forms a local volcanic group with Chachani to the northwest and Pichu Pichu to the southeast, built on debris from older volcanoes. Over the past 50,000 years, numerous intense explosive eruptions have covered the surrounding terrain with tephra (rocks fragmented by volcanic eruptions). The last two significant eruptions were 2,000 years ago and in 1440–1470 AD; since then, periods of increased gas release have sometimes been mistaken for eruptions.
Misti is considered one of the most dangerous volcanoes in the world, as it lies only 12 km from Arequipa. The city's population exceeds one million people and its northeastern suburbs have expanded onto the slopes of the volcano. Narrow valleys on its western and southern flanks pose particular threats, as mudflows and fast-moving flows of hot volcanic debris could be channelled into the urban area and into important infrastructure, like hydropower plants. Even moderate eruptions can deposit volcanic ash and tephra over most of the city. Until 2005, there was little awareness or monitoring of the volcano. Since then, the Peruvian Geological, Mining and Metallurgical Institute (INGEMMET) has established a volcano observatory in Arequipa, runs public-awareness campaigns, and has published hazard maps. The Inca viewed the volcano as a threat and during the 1440–1470 eruption offered human sacrifices (capacocha) on Misti and neighbouring mountains to placate the volcano; the mummies on Misti form the largest known Inca human sacrifice.
Name and settlement history
The name "Misti" may derive from either Quechuan or Spanish. It means 'mixed', 'mestizo' or 'white' and may refer to the volcano's snow cover. Indigenous names include Putina,[1][2] meaning 'mountain that growls'[3] in the Puquina language, while the Aymara language terms for Misti are Anukara[4] or Anuqara[5] ('dog'). All three names refer to the dog-like appearance of the volcano when viewed from the Andean Plateau, known as the Altiplano.[3] Earlier sources referred to the volcano as Putina; the name "Misti" appears from the 1780s onward.[6] Other names for the volcano are Guagua-Putina, El Volcán ('the volcano'), San Francisco, and Volcán de Arequipa ('Arequipa
Characters
3606
Words
560
Approx. Tokens
945
Generate Summary


Generated Summary
Misti is a volcano in the southern Peruvian Andes, rising above Arequipa. It has a steep conical profile and two nested summit craters. Snow covers the summit but does not persist, and there are no glaciers. Peruvian Geological, Mining and Metallurgical Institute (INGEMMET) has established a volcano observatory in Arequipa. The Inca viewed the volcano as a threat and during the 1440–1470 eruption offered human sacrifices on Misti and neighbouring mountains. Indigenous names include Putina, [1][2] meaning'mountain that growls'[3] in the Puquina language, while the Aymara language terms for Misti are Anukara It is unclear whether the Inca were the first Altiplano civilizations to influence the region or whether previous cultures played a role. By the arrival of the Spanish, the area was densely populated,[11] There were canals, roads and buildings where the city of Arequipa is today.
Summary Statistics
Summary Words
138
Chunks Processed
3
Summary / Original
24.64%
Model Information
Evaluation Metrics
Training and Evaluation Plots


Summary Statistics
Summary Words
138
Chunks Processed
3
Summary / Original
24.64%
Model Information
Model: T5-small
Dataset: CNN/DailyMail
Task: Abstractive Text Summarization
Input limit per T5 call: 512 tokens
Output limit: 128 tokens
Device: cuda


Training and Validation Loss
Training Loss
Validation Loss
Loss
Training Steps
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0
500
1000
1500
2000
2500


Final Test ROUGE Scores
Score
0.30
0.25
0.20
0.15
0.10
0.05
0.00
ROUGE-1
ROUGE-2
ROUGE-L

CONCLUSION

The **"Text Summarization Using Deep Learning"** project successfully demonstrates the use of the **T5-small Transformer model** for abstractive text summarization. The model is fine-tuned using the **CNN/DailyMail dataset** to generate concise summaries while retaining the important information and context of the original text.The **Transformer model was chosen instead of an LSTM model** because Transformers use a **self-attention mechanism**, which allows the model to focus on important words and capture relationships between words even when they are far apart in a document. LSTM models process text sequentially, which can make training slower and make it more difficult to capture long-range dependencies in lengthy documents. Transformers can process input tokens in parallel during training, providing better efficiency and making them well suited for modern text-generation tasks. The **encoder-decoder architecture of T5** is particularly appropriate for summarization because the encoder understands the input article and the decoder generates the corresponding summary.The project includes **data preprocessing, tokenization, model fine-tuning, summary generation, and performance evaluation** using ROUGE metrics along with supplementary performance measures. A **Streamlit-based interface** provides an easy way for users to enter text and obtain generated summaries, while chunking allows longer documents to be processed.Overall, the project demonstrates that **Transformer-based Deep Learning models provide a suitable and efficient approach for abstractive text summarization**, helping reduce the time and effort required to understand large volumes of textual information.